

Calcolatori Elettronici T
Ingegneria Informatica

01 Introduzione



Docenti

Stefano Mattoccia

Ricevimento: appuntamento concordato via email

Email: stefano.mattoccia@unibo.it

Home page: <http://vision.disi.unibo.it/~smatt>

Ugo Leone Cavalcanti (tutor)

Ricevimento: appuntamento concordato via email

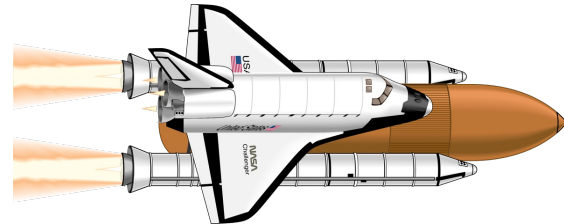
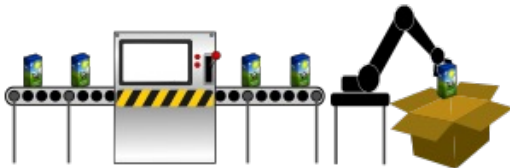
Email: ugoleone.cavalcanti@unibo.it

Obiettivi del corso

Apprendere,

- i principi di funzionamento
- le architetture
- la progettazione hardware e software

dei sistemi per l'elaborazione delle informazioni basati
microprocessore (o CPU - Central Processing Unit)



Orario lezioni

Orario ufficiale:

<https://corsi.unibo.it/laurea/IngegneriaInformatica/orario-lezioni?anno=2&curricula=>

Lunedì 14.00-16.00, inizio ore **14.00** Aula **6.1**

Giovedì 14.00-17.00, inizio ore **14.00** Aula **6.1**

Materiale didattico

Lucidi in formato PDF

<https://virtuale.unibo.it>

Comunicazioni attraverso piattaforma Virtuale.

Prove d'esame

<https://virtuale.unibo.it>

I lucidi non sono un libro, è opportuno seguire con attenzione le lezioni

Materiale didattico per approfondimenti

Seguendo il corso con attenzione e utilizzando i lucidi forniti **NON è necessario utilizzare altro materiale** per la preparazione dell'esame.

Tuttavia, per chi desidera approfondire:

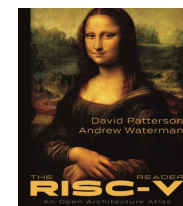
- D. Hennessy, J. Patterson, "Computer architecture: a quantitative approach", Morgan Kaufmann
Versione italiana. La seconda edizione (inglese, a dx) descrive in dettaglio il processore DLX



- D. Patterson, J. Hennessy, "Struttura e progetto dei calcolatori, Progettare con RISC-V", Zanichelli



- Patterson, Waterman, "The RISC-V Reader", Strawberry Canyon



Requisiti (consiglio)

Per superare in modo proficuo il corso di Calcolatori Elettronici T è fondamentale avere compreso bene:

- 1) **Fondamenti di Informatica T**
- 2) **Reti Logiche T**

In particolare, per Reti Logiche T è cruciale avere appreso la modalità di progettazione *diretta*.

Si sconsiglia vivamente di seguire questo corso senza avere solide basi in 1) e **soprattutto di Reti Logiche T**

Modalità di svolgimento dell'esame

- L'esame consiste in due prove:
 - Una prima parte (peso 1/3): scritta con domande sui contenuti del corso ed esercizi: durata 30 min
 - Una seconda parte (peso 2/3) di progetto di un sistema a microprocessore: durata 2 ore
- Se la prima parte è insufficiente, la seconda parte non sarà considerata
- In entrambi i casi non è possibile utilizzare libri, appunti, computer, telefoni, smartphone, tablet, smart watch, etc
- E' indispensabile (pena l'esclusione dall'esame) presentarsi con documento di identità e badge
- Tutto sarà verbalizzato

Esempio di prova d'esame

ESAME DI CALCOLATORI ELETTRONICI T (Ing. Informatica) 05/09/2024
COGNOME _____ NOME _____ MATRICOLA _____

Domanda 1

Quale valore assume il registro R1 al termine dell'esecuzione del codice seguente?

```
ADDI R1,R0,0xFF
SLI  R2,R1,0x0004
OR   R3,R1,R1
SLI  R1,R3,0x0008
R1 = 0x_____
```

Domanda 2

Nel DLX, come avviene l'estensione di un byte *signed* letto dalla memoria? Spiegare in dettaglio.

Domanda 3

E' possibile utilizzare con il DLX un PC di dimensione inferiore a 32 bit?
[SI, perchè?] [NO, Perchè]

Domanda 4

In un sistema reale basato sul DLX è sempre necessaria la rete logica che consente di determinare se il sistema è in fase di avvio?
[SI, Perchè?] [NO, Perchè?]

Domanda 5

Sarebbe possibile gestire due richieste di interruzione asserite contemporaneamente mediante il codice di un unico interrupt handler?
[SI, Come?] [NO, Perchè?] - Motivare la risposta

Risposte senza corrette e mirate motivazioni saranno considerate completamente errate

Esame di Calcolatori Elettronici T 5 Settembre 2024 (Ing. Informatica)

Esercizio 1

Progettare un sistema, basato su un processore DLX dotato di 2 GB di EPROM mappata agli indirizzi bassi e 1 GB di RAM mappata a partire dall'indirizzo 80000000h.

Nel sistema sono presenti quattro porte in input: INPUT_0, INPUT_1, INPUT_2, INPUT_3. Le porte in input sono già progettate, abilitate in modo mutuamente esclusivo, e ciascuna in grado di trasferire 8 bit mediante il protocollo di *handshake*. All'avvio è attiva INPUT_0. Ad ogni lettura da INPUT_0 dovrà seguire una lettura da INPUT_1, poi una lettura da INPUT_2 e successivamente una lettura da INPUT_3 e così via (i.e., INPUT_0 -> INPUT_1 -> INPUT_2 -> INPUT_3, etc). I byte letti da ciascuna porta dovranno essere scritti all'indirizzo 80000000h.

L'alternanza tra le quattro porte, in accordo a quanto delineato in precedenza, dovrà essere eseguita mediante l'utilizzo di opportune reti logiche e NON unicamente via software.

- Descrivere sinteticamente la soluzione indicando chiaramente quali sono i dispositivi utilizzati, gli indirizzi e i segnali di *chip-select*
- Progettare il sistema indicando le espressioni di decodifica e il range di indirizzi di tutte le periferiche, le memorie e i segnali e le connessioni di tutti i dispositivi con i bus di sistema. Evidenziare eventuali criticità.
- Scrivere il codice dell'interrupt handler assumendo che i registri da R20 a R25 non debbano essere ripristinati

30 min, peso 1/3
if (Prova_1<18) fine
else -> Prova_2

2 ore, peso 2/3

Voto finale: Prova_1*1/3 + Prova_2*2/3

Prossimi appelli d'esame

Se possibile (aule, etc) il primo appello subito dopo il termine delle lezioni

- La date degli esami saranno consultabili in Almaesami
- L'iscrizione agli appelli via Almaesami è obbligatoria e si chiude (circa) una settimana prima
- Non è ammessa alcuna deroghe all'iscrizione
- Sono previsti 6 appelli all'anno:
 - 3 Gennaio/Febbraio
 - 2 Giugno/Luglio
 - 1 Settembre

Nessun appello straordinario

E' possibile rifiutare una sola volta un esito positivo

Elaborazione delle informazioni

Un essere vivente elabora continuamente delle informazioni e fornisce delle risposte o agisce in un determinato modo in base alle informazioni in ingresso.



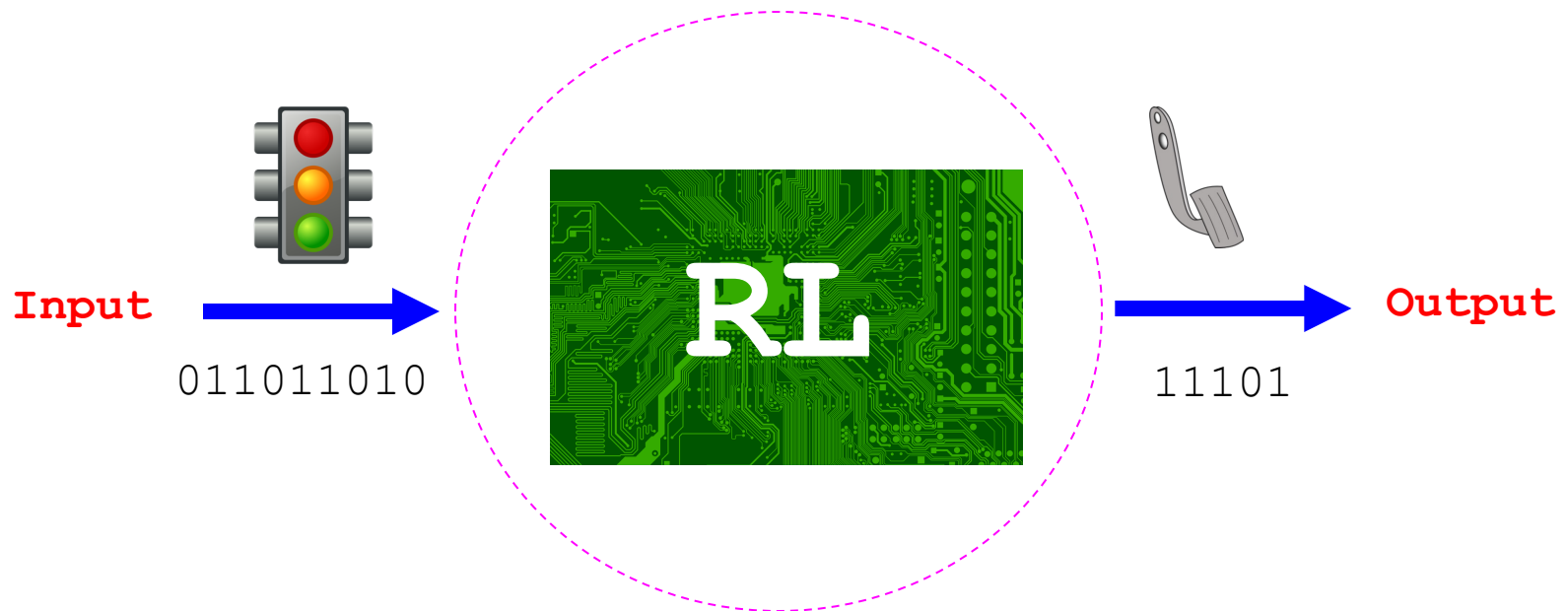
Un sistema di qualsiasi natura per l'elaborazione delle informazioni isolato dall'esterno servirebbe a ben poco (meglio, nulla).

Input e output debbono essere codificati in modo appropriato

Elaborazione delle informazioni

Ovviamente, in questo corso, siamo interessati a sistemi di natura elettronica per elaborare le informazioni.

In particolare, per le ragioni evidenziate a Reti Logiche T, siamo interessati a **sistemi digitali** -> Reti Logiche



Elaborazione delle informazioni

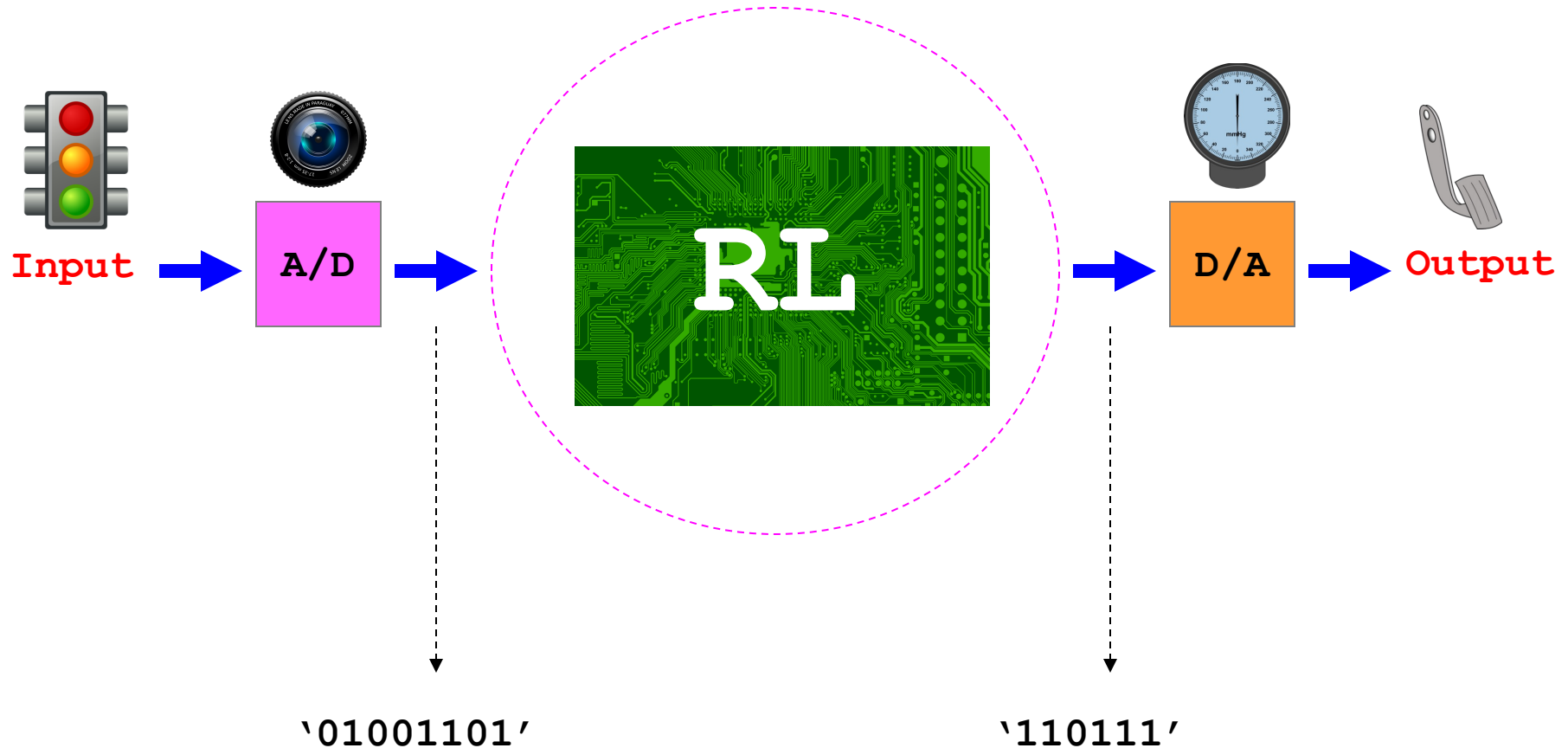
Un primo problema: le reti logiche elaborano informazioni di tipo digitale

Molti input e output di particolare interesse non sono di natura digitale ma analogica (quella preferita dagli esseri umani)

- Prossimità a un semaforo o un ostacolo
- Monitor
- Movimento del mouse
- Voce/Audio
- Pressione tasti
- Touch screen
- etc

Elaborazione delle informazioni

Pertanto, è (spesso) necessario convertire segnali di natura analogica in segnali digitali e viceversa



In un sistema di elaborazione, in ingresso e in uscita troviamo solo segnali digitali (binari)

Elaborazione delle informazioni

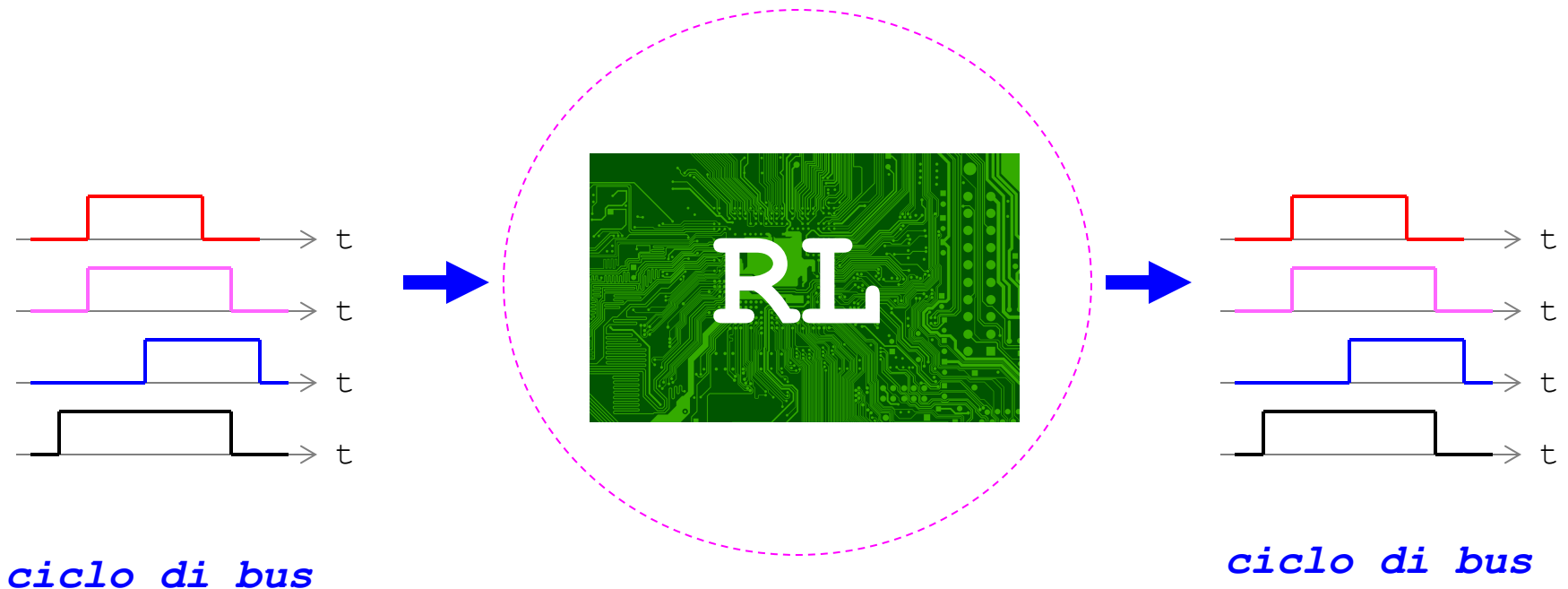
Altro problema: la rete logica che elabora le informazioni è spesso molto efficienti/veloce/etc ma allo stesso tempo poco flessibile per quanto riguarda il *protocollo* utilizzabile per fornire le informazioni (digitali) in input e *interpretare* le informazioni (digitali) elaborate in output.

Se si desidera interagire con un sistema di elaborazione digitale e necessario adeguarsi al *protocollo* che la rete logica utilizza.

Questo protocollo è codificato dall'evoluzione temporale di alcuni segnali (il minimo indispensabile) emessi o ricevibili dalle reti logiche coinvolte. L'evoluzione temporale di questi segnali si definisce *ciclo di bus* (*bus cycle*).

Elaborazione delle informazioni

Quindi, **l'unico modo per interagire** con un sistema digitale per l'elaborazione delle informazioni consiste nell'adottare questa convenzione (ie, i cicli di bus, descritti in dettaglio sui *datasheet* che il produttore del sistema rende sempre disponibili).



Quale rete logica utilizzare?

Sono disponibili diverse architetture di elaborazione (i.e., reti logiche). Quale scegliere?

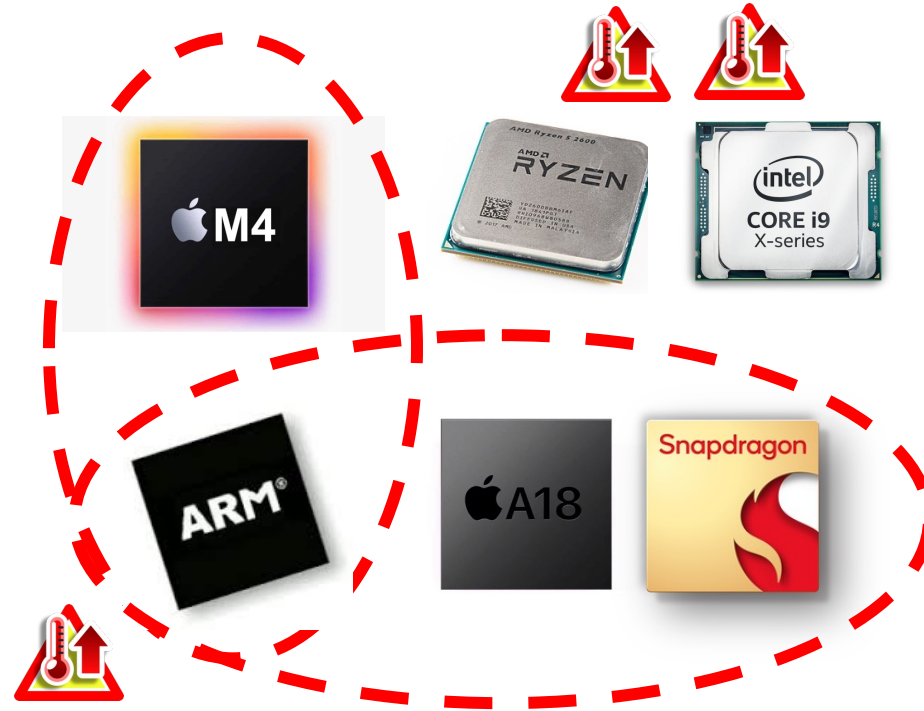
Dipende dal contesto applicativo e dalle prestazioni, consumi, ingombri, peso, etc:

- sistema *general purpose*
- sistema embedded
- sistema per videogiochi
- etc

Una tipologia di architettura, basata sul **modello di Von Neumann**, è molto più flessibile delle altre e può essere utilizzata, anche se non sempre con risultati ottimali, in ogni contesto. Questo modello è l'oggetto di questo corso e utilizza come elemento di base una CPU (microprocessore).

Esempi di architetture di elaborazione

- CPU
 - CPU Multicore
 - CPU Embedded
- SOC (System on a Chip)



- GPU (Graphic Processing Unit)
- FPGA (Field Programmable Gate Array)
- Sistemi ibridi (e.g. FPGA + CPU)

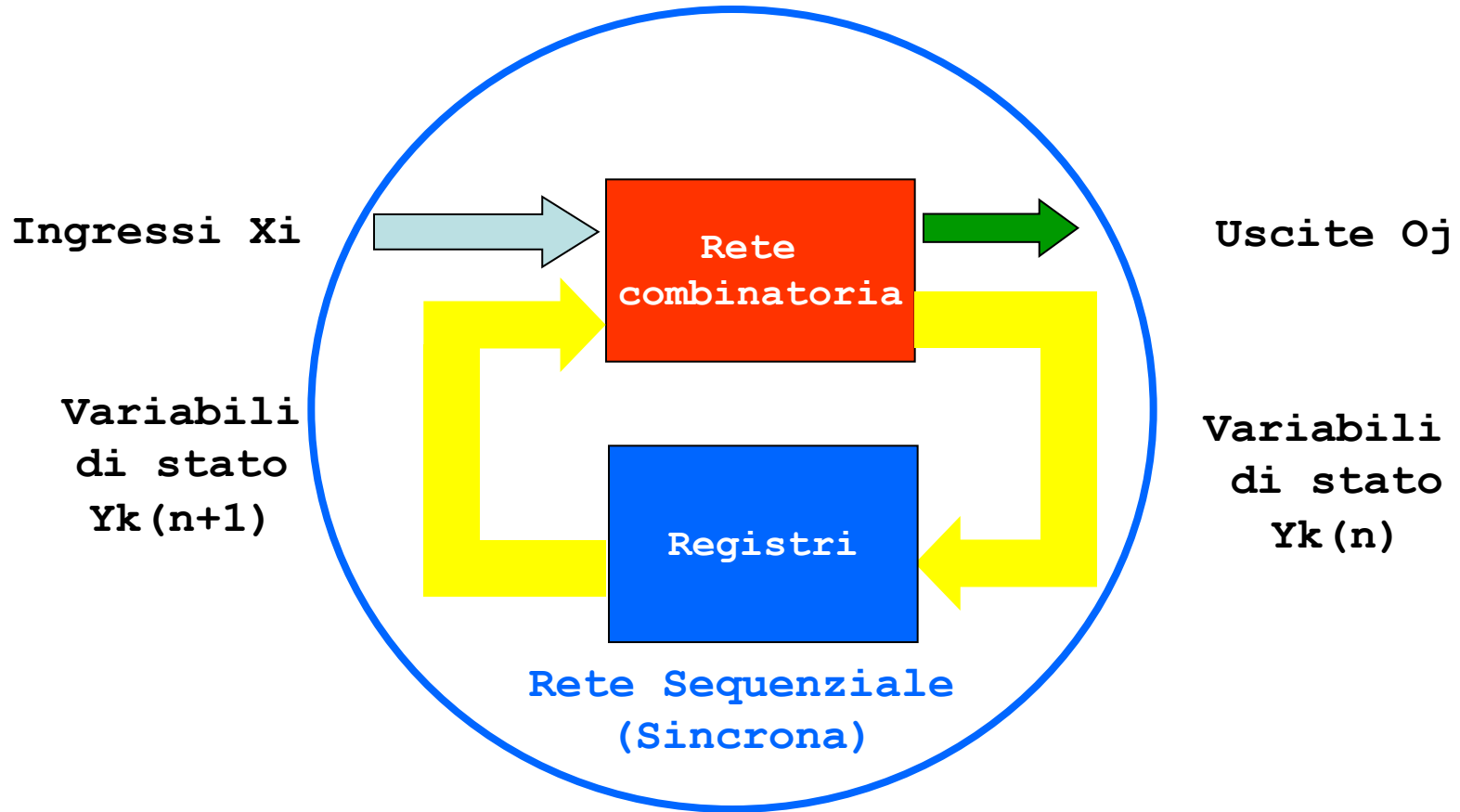


Il modello di riferimento: Von Neumann

- Architettura del tutto generale che porta a realizzazioni poco dipendenti dal funzionamento desiderato
- Il funzionamento desiderato è espresso in termini di
 - * sequenza di istruzioni (programma)
 - * memorizzate su un supporto di memoria
- Per cambiare funzionamento è sufficiente cambiare il programma: questo di fatto “modifica” la rete logica di controllo per ogni istruzione eseguita
- L'architettura è adatta a trattare problemi molto più complessi di quelli visti nel corso di reti logiche ma con efficienza (tipicamente) inferiore
- L'importanza e la diffusione dei calcolatori dipende fortemente dalla *flessibilità* di questo modello

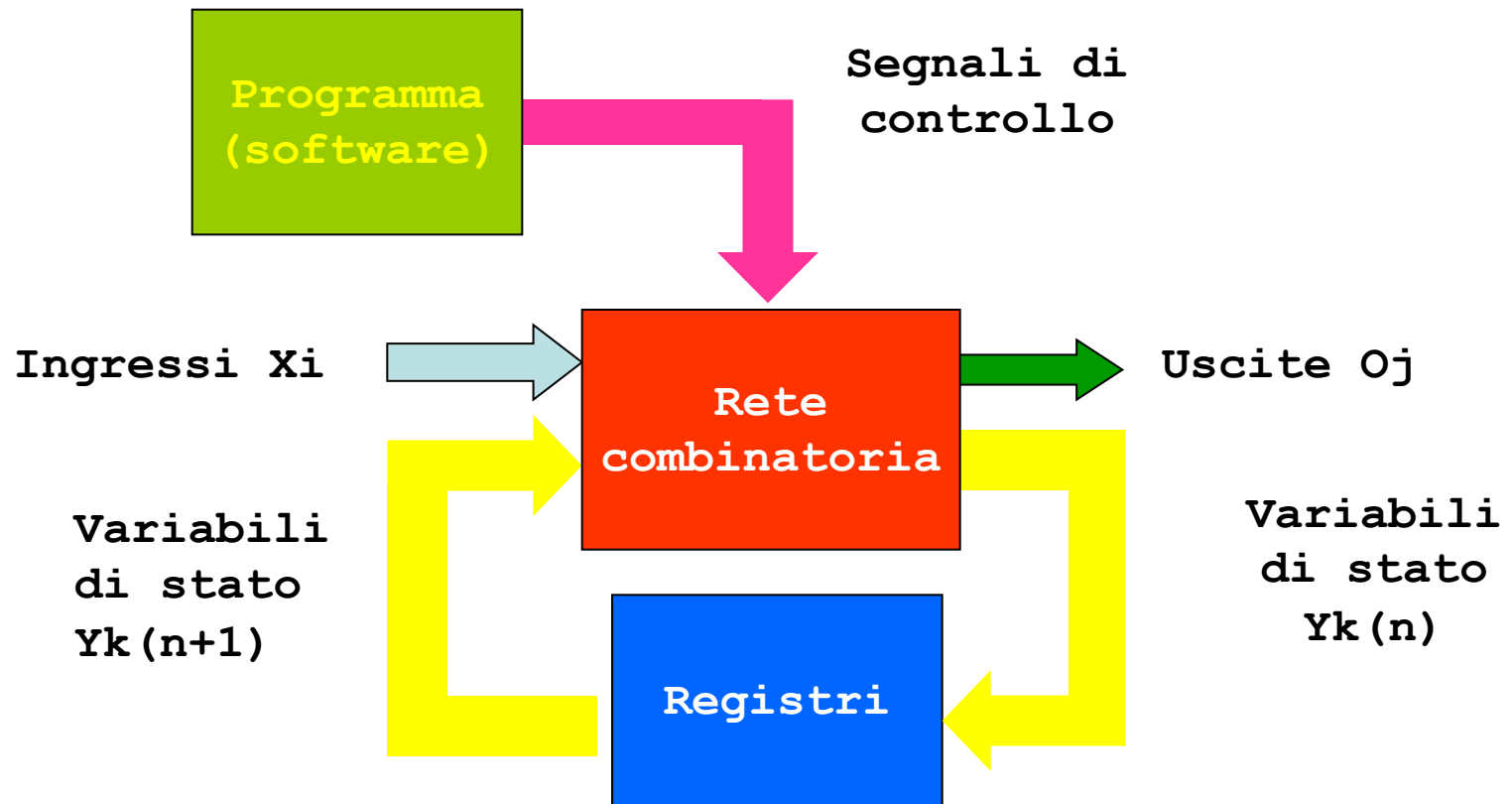
Sistema di elaborazione: rete sequenziale

Il sistema di elaborazione può essere schematizzato in modo astratto come una Rete Sequenziale (Sincrona), RSS

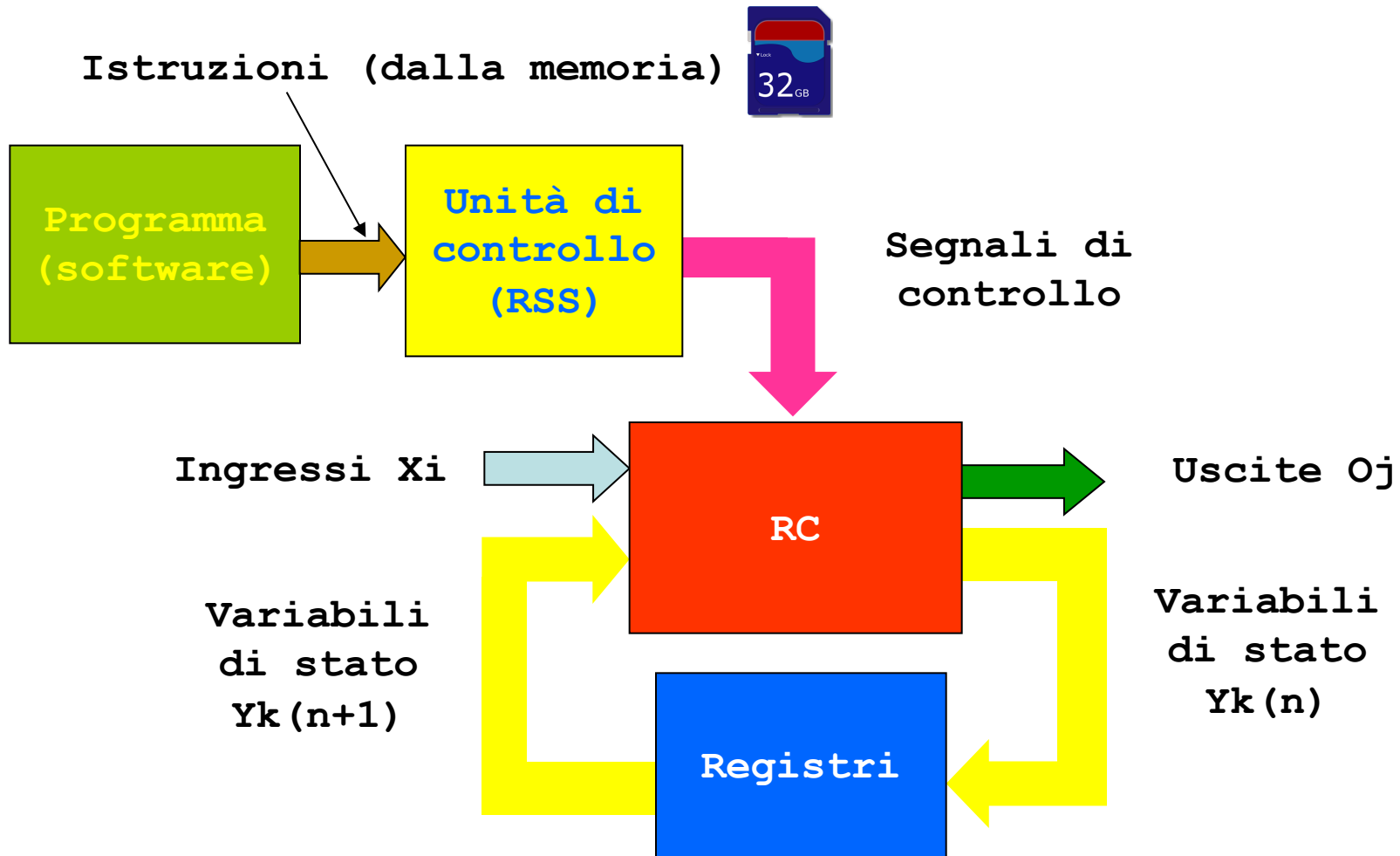


Come cambiare il funzionamento della RL?

Rispetto a Reti Logiche, la novità è il **software** (programma) che consente di variare il funzionamento della rete in base alle esigenze desiderate (ie, il codice scritto dal programmatore).



In realtà le cose sono più complesse

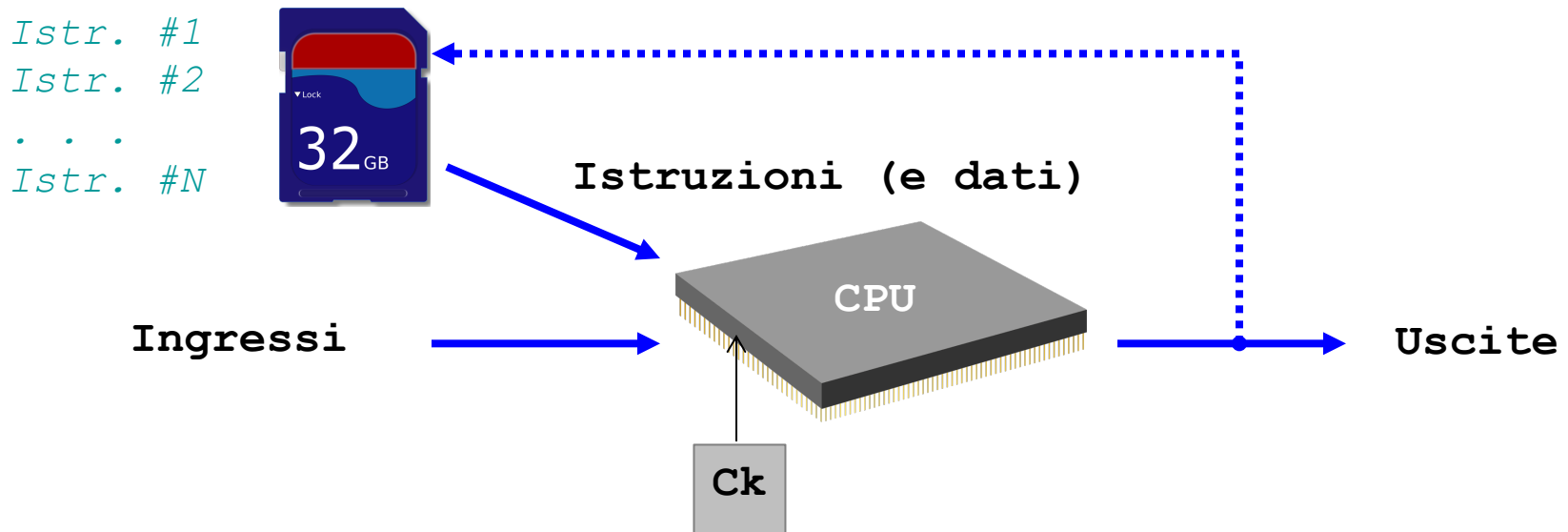


L'unità di controllo non solo governa la rete combinatoria ma anche tutte le altre reti logiche presenti nel sistema

Esempio: abilita gli ingressi e le uscite quando necessario, etc

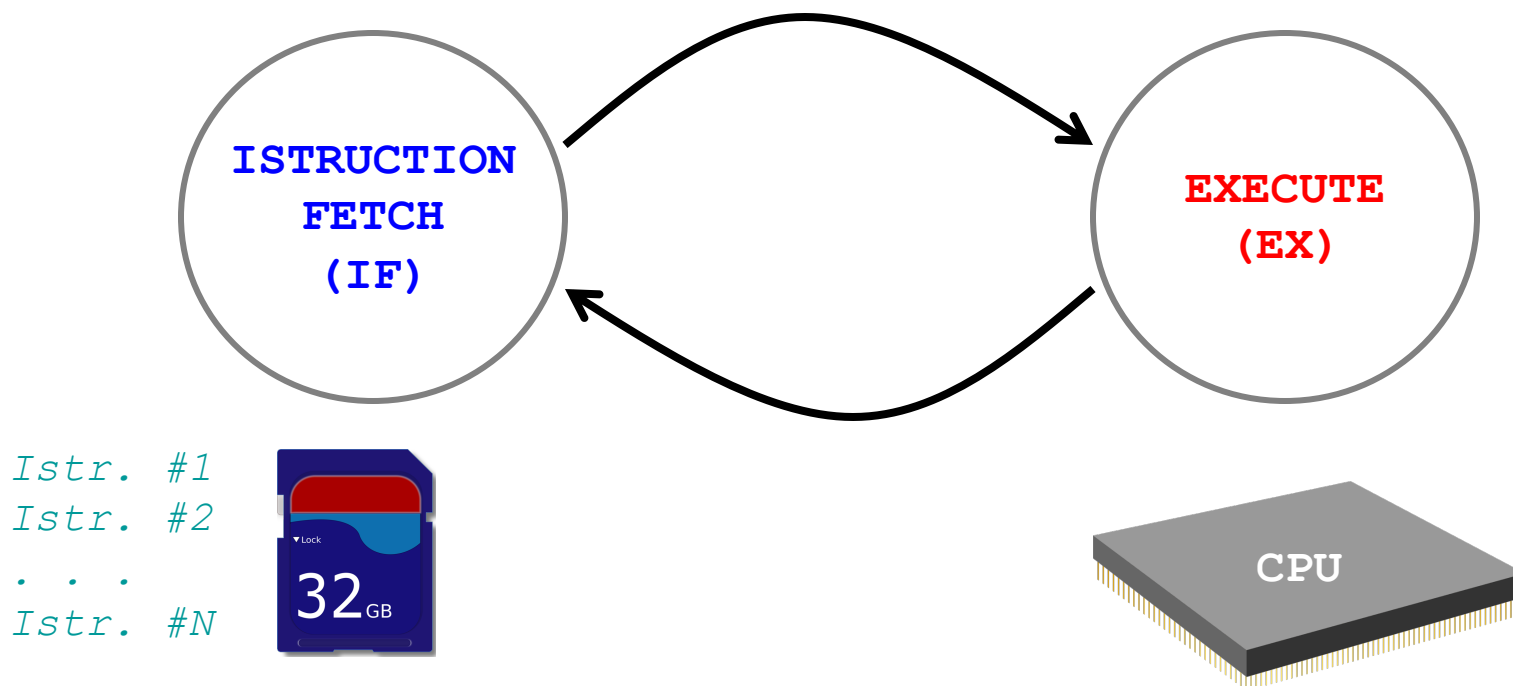
Modello di esecuzione del programma

- Il programma risiede in memoria ed è costituito da istruzioni codificate in forma binaria (linguaggio macchina)
- In memoria risiedono anche gli operandi delle istruzioni, cioè i dati elaborati e da elaborare (forma binaria)
- Le istruzioni vengono eseguite in sequenza dalla CPU
- La CPU è una macchina sequenziale sincrona (con un clock)



A livello di massima astrazione, il funzionamento dell'intero sistema può essere descritto mediante due soli stati:

- Stato in cui la CPU legge in memoria la prossima istruzione da eseguire (**INSTRUCTION FETCH** o IF)
- Stato in cui la CPU esegue l'istruzione letta in IF (**EXECUTE** o EX)



RL CPU e istruzioni

- Le reti viste a Reti Logiche T elaboravano e producevano segnali binari
- Se la CPU è una RL, come sono codificate le istruzioni?
- Ovviamente in binario...

- Esempio

istruzione #1	->	00010100000101111101010000010011
istruzione #2	->	10110101100100011001010000011001
. . .		. . .
istruzione #N	->	0101011010010101010101010110011110

- Ogni istruzione indica alla CPU quale operazione deve svolgere/eseguire
- Non sembra essere un modo molto comodo (per gli esseri umani)

Esempi di istruzioni

- Quali istruzioni possono essere eseguite da una CPU?
 - Somme
 - Moltiplicazioni
 - Divisioni
 - Confronto
 - Letture dalla memoria o da altri dispositivi*
 - Scritture in memoria o verso altri dispositivi*
 - . . .

Esistono sostanzialmente due tipologie di CPU:

- **RISC** (Reduced Instruction Set Computer)
Poche e semplici istruzioni, reti logiche semplici e molto veloci (frequenza di clock elevata). eg, ARM
- **CISC** (Complex Instruction Set Computer)
Molte istruzioni, alcune molto complesse, reti logiche complicate. eg, Intel e AMD

RISC vs CISC

- Tipicamente a una sola istruzione CISC corrispondono più istruzioni RISC
- Tuttavia, ogni singola istruzione RISC è eseguita spesso più rapidamente di una istruzione CISC
- Spesso, il codice RISC, anche se più *denso*, è più veloce
- Le RL RISC sono tipicamente più semplici di quelle CISC
- Se le reti sono più semplici lo spazio (silicio) può essere utilizzato per altre finalità (registri, *cache*, etc)
- I processori RISC sono molto diffusi (smartphone, tablet, e ora anche in alcuni computer standard (eg, Apple))
- Anche i processori CISC sono molto diffusi (PC) per via del software esistente*
- Le CPU CISC moderne sono in realtà internamente dei RISC

Istruzioni in forma più comprensibile

- Indipendentemente dal tipo di CPU, le istruzioni in forma binaria non sono facilmente interpretabili e per questo non utilizzate in questa forma quando si scrive il codice
- Il linguaggio che si utilizza è l'**assembly** o **assembler**:

`ADD R1,R2,R3;` pone in R1 la somma tra registri R2 e R3

Questa istruzione potrebbe essere codificata con:

00010100000101111101010000010011

Il traduttore assembler -> codice macchina in binario è una Look Up Table (LUT), ovvero una tabella

L'assembler sembra essere un passo avanti notevole ma...

Perché non avete mai utilizzato questo linguaggio?

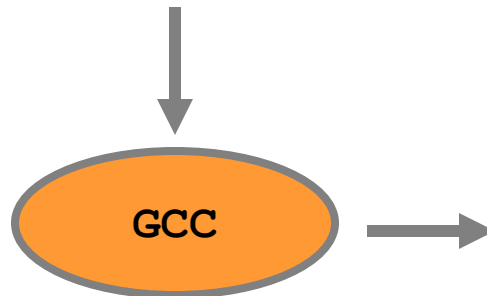
Compilatore e istruzioni binarie

- Il motivo è che avete scritto codice ad alto livello (C) e utilizzato un compilatore (e.g., GCC)
- Il compilatore, converte il codice scritto in linguaggio ad alto livello in istruzioni macchina binarie (bypassando la necessità di scrivere codice assembly)

```
#include <stdio.h>
int main(int argc, char **argv)
{
    int a=5;
    int b=6;
    int c;

    c = a + b;

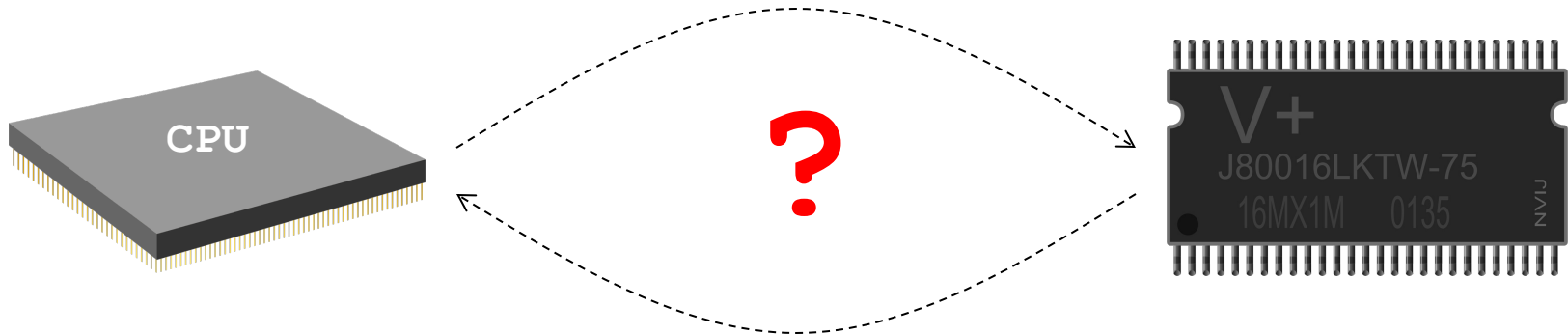
    printf("La somma tra %d e %d è %d\n", a, b, c);
}
```



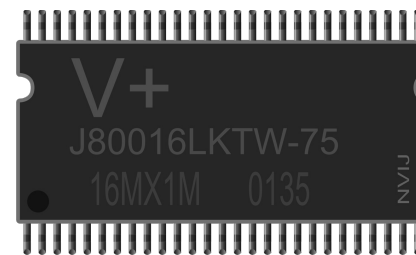
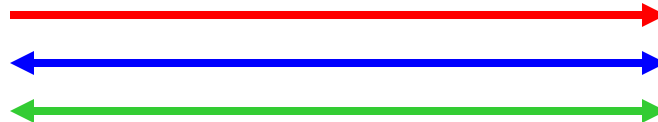
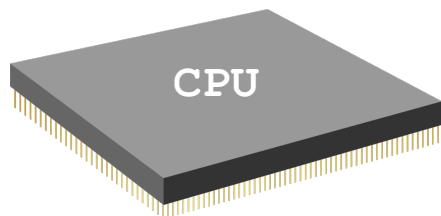
```
00010100000101111101010000010011
10110101100100011001010000011001
. . .
01010110100101010101010110011110
```

Come si accede alla memoria (e non solo)?

- Sappiamo che il codice nel modello di Von Neumann risiede in memoria
- Come si legge (e scrive dalla memoria)?
- Come si leggono e scrivono i dati?



- L'unico modo è attraverso dei segnali predefiniti (con un ben definito andamento temporale, **ciclo di bus**)



Indirizzi BA[K..0]



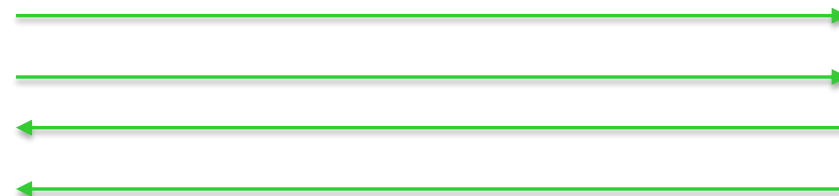
Dati

BD[R..0]



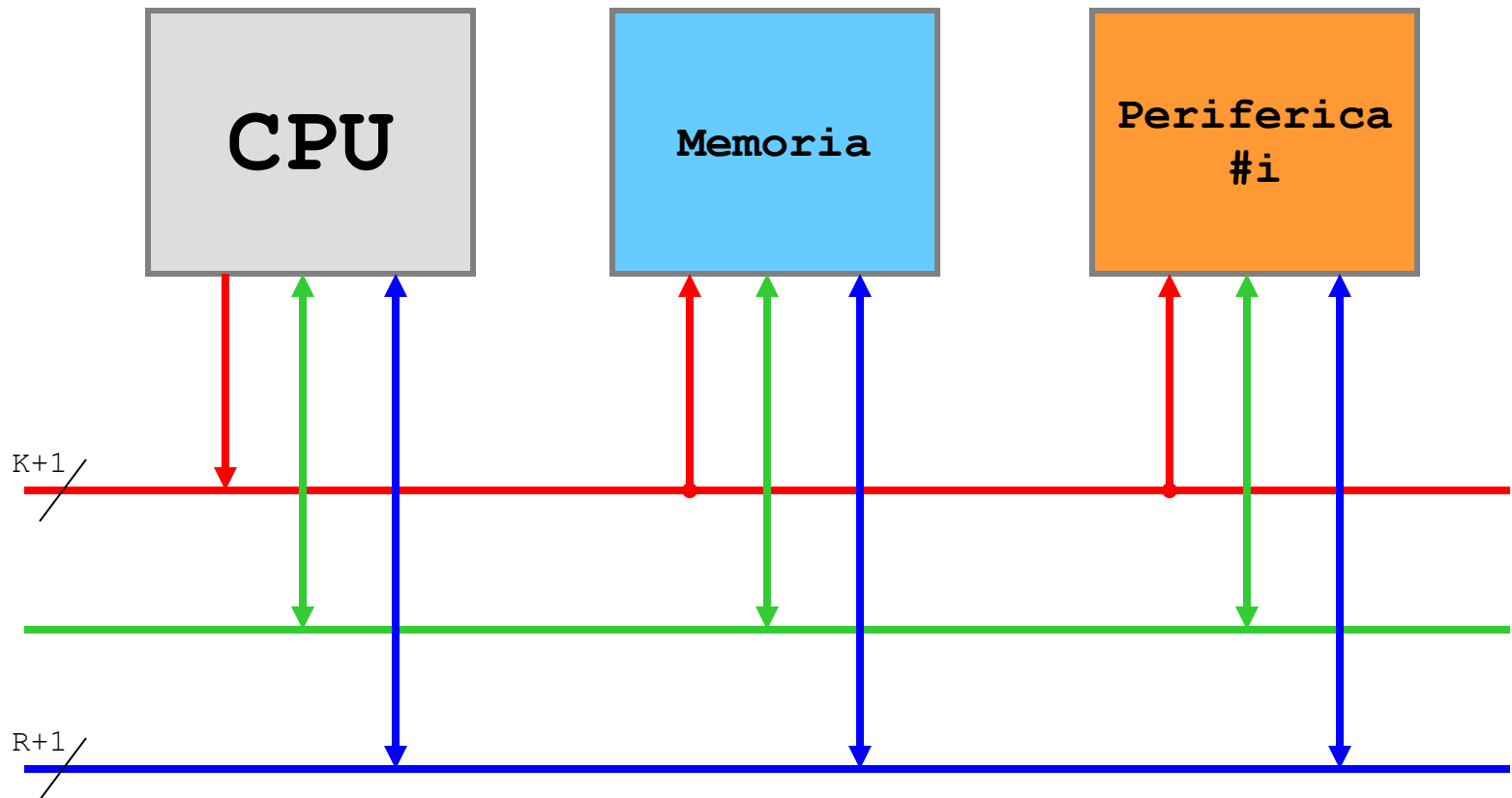
Controllo

{
 READ
 WRITE
 READY
 INT

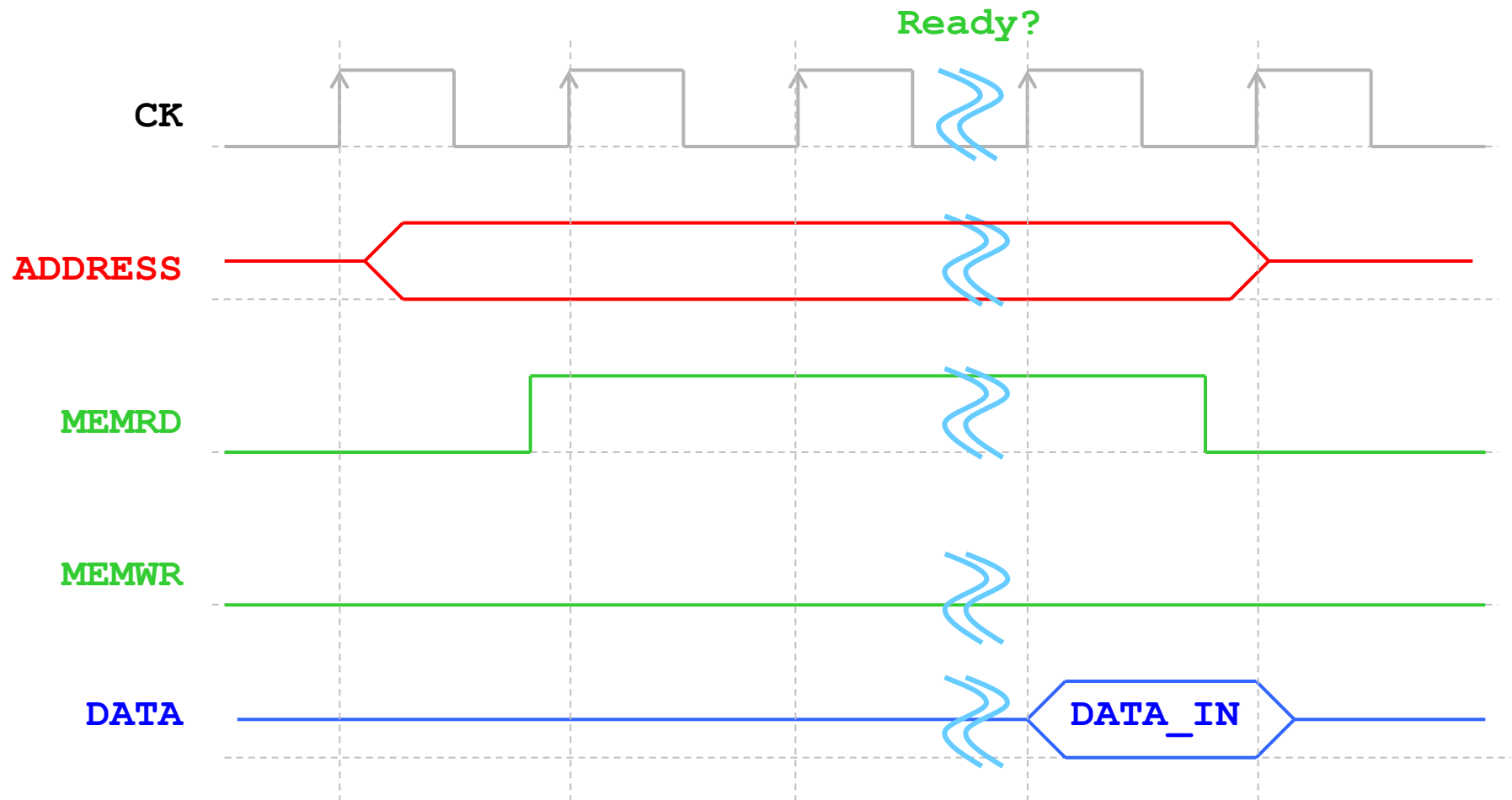


Come avviene la comunicazione?

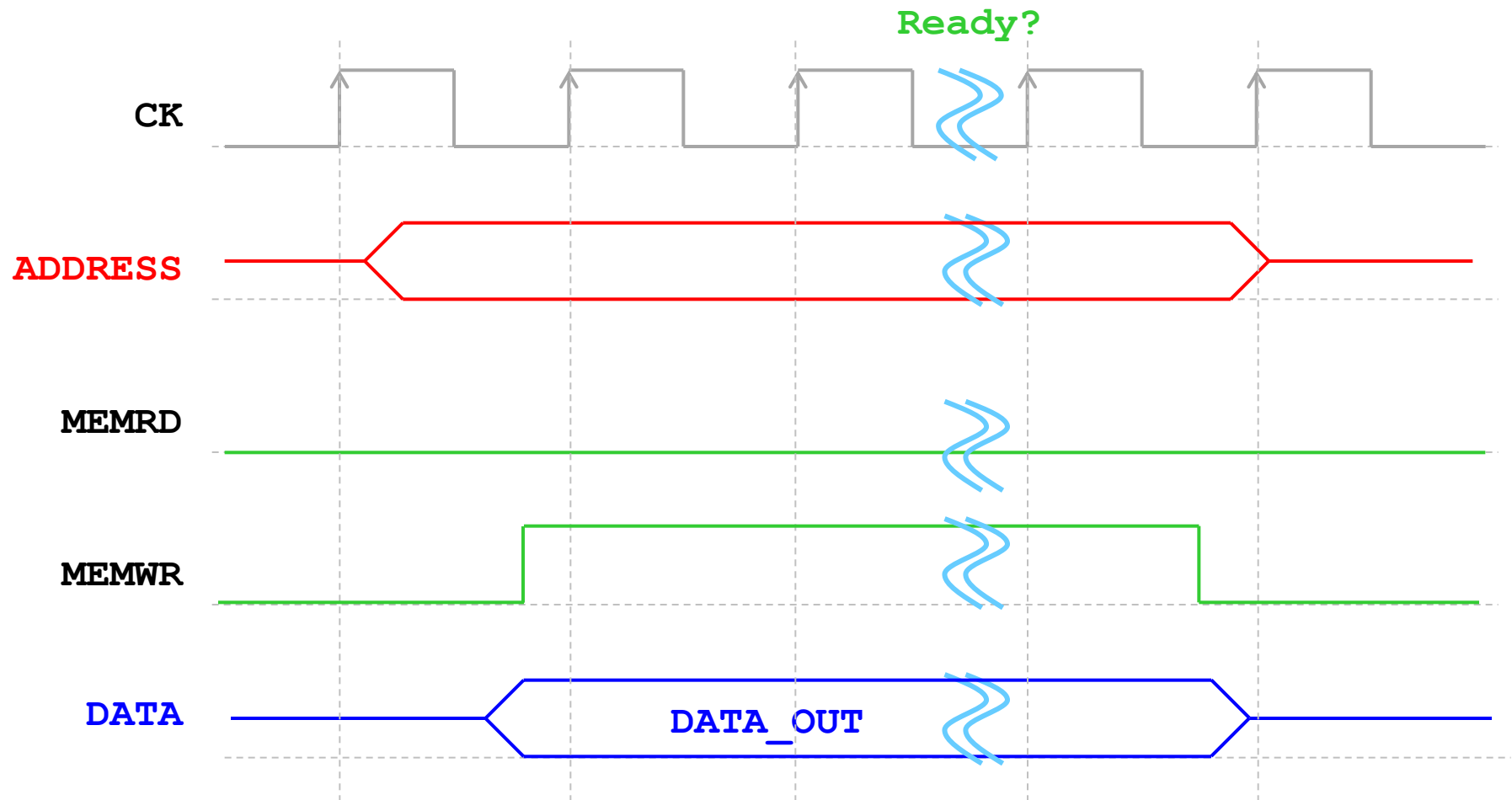
- Tutto viaggia sui bus di sistema
- Tutto è regolato da cicli di bus



Esempio di ciclo di lettura



Esempio di ciclo di scrittura



Range di indirizzi

- Se sono disponibili 32 bit (**BA[31..0]**) è possibile avere accesso a 2^{32} elementi (memorie, periferiche)
- 32 segnali di indirizzo, 4 GB
- A ogni indirizzo è associato un dispositivo (memorie, periferiche)
- E' necessario decodificare l'indirizzo emesso dalla CPU per determinare con quale dispositivo la CPU intende comunicare
- Quanti byte possono essere trasferiti durante un ciclo di bus? Dipende dall'ampiezza del bus dati

Relazione tra hardware e software

- Consideriamo una istruzione per leggere dalla memoria un byte a un determinato indirizzo

xxx **LB** **destinazione**, **indirizzo** ; lettura di un byte

Per prima cosa è eseguito il fetch dell'istruzione all'indirizzo **xxx**. come? Con un ciclo di bus, naturalmente

Come facciamo a conoscere l'indirizzo **xxx**?

La CPU ha accesso al **program counter** **PC**

Una volta letta e decodificata, l'istruzione è eseguita.

Durante l'esecuzione è eseguito un ciclo di bus di lettura

Durante il ciclo di bus è emesso l'**indirizzo**